Clasificación Automática de Quejas Financieras de Consumidores Mediante Deep Learning

Yuri Biardo | Andrea Aranda | Mariana Sambucetti

Técnicas de Inteligencia Artificial: Deep Learning — Febrero 2026

1. Introducción

El CFPB (Consumer Financial Protection Bureau) de Estados Unidos recibe a diario una cantidad enorme de quejas de consumidores sobre productos financieros. Clasificar todas esas quejas a mano lleva mucho tiempo y sale caro, y encima el lenguaje que usa cada persona es muy variado, lo que complica las cosas para cualquier enfoque que no sea de aprendizaje automático.

Lo que buscamos con este proyecto fue armar un clasificador automático que pueda tomar el texto de una queja y asignarle la categoría de producto correcta. Trabajamos con 5 categorías: *credit_card, credit_reporting, debt_collection, mortgages_and_loans* y *retail_banking*. Para resolver esto usamos Deep Learning, puntualmente hicimos fine-tuning de DistilBERT, un modelo basado en Transformers.

2. Solución propuesta

Elegimos **DistilBERT** (distilbert-base-uncased) como modelo base. Es una versión más chica de BERT: tiene 66 millones de parámetros en vez de 110 millones, pero mantiene el 97% del rendimiento. La ventaja principal para nosotros es que cómodamente se puede usar con Google Colab.

DistilBERT está construido sobre la arquitectura **Transformer**, que usa un mecanismo de **self-attention** para procesar el texto. La gracia de esto es que el modelo puede mirar todas las palabras de una oración al mismo tiempo y entender las relaciones entre ellas, algo que los modelos secuenciales como RNN o LSTM no hacen tan bien porque procesan las palabras de a una.

El concepto detrás de lo que hicimos se llama **fine-tuning**: tomamos un modelo que ya aprendió a entender texto (porque fue pre-entrenado con millones de textos de internet) y lo adaptamos a nuestra tarea puntual. En la práctica, le pusimos una capa de clasificación con 5 salidas al final y lo entrenamos con nuestros datos. Para el tema del desbalance de clases (que mencionamos más adelante), armamos un WeightedTrainer que le da más importancia a las categorías con menos datos.

2.1 Hiperparámetros

Hiperparámetro	Valor	Por qué
Epochs	3	Alcanza para que converja sin sobreajustar
Batch size	16	Se puede correr bien en Colab
Learning rate	0,00002	Es el valor habitual para fine-tuning de BERT
Max length	256 tokens	Ahí entra el 94.6% de las quejas
Weight decay	0.01	Regularización para evitar sobreajuste

3. Datos

Usamos el **Consumer Complaints Dataset for NLP** de Kaggle, con 162.411 quejas reales del CFPB (Consumer Financial Protection Bureau). Cada registro tiene la narrativa (texto libre que escribió el consumidor) y la categoría de producto.

3.1 Distribución

El dataset tiene un desbalance bastante marcado. Credit_reporting acapara más de la mitad de los datos:

Categoría	Cantidad	Porcentaje	Peso
credit_reporting	91.179	56.14%	Bajo
debt_collection	23.150	14.25%	Medio
mortgages_and_loans	18.990	11.69%	Medio-alto
credit_card	15.566	9.58%	Alto
retail_banking	13.536	8.33%	Alto

Para que el modelo no se vicie prediciendo siempre credit_reporting, usamos pesos de clase inversamente proporcionales a la cantidad de datos en la función de pérdida (CrossEntropyLoss). Así las categorías chicas pesan más cuando el modelo se equivoca.

3.2 Preprocesamiento

Primero sacamos las filas con valores vacíos en las columnas de texto o etiqueta. Después tokenizamos con el tokenizador propio de DistilBERT, que parte el texto en sub-palabras y les asigna IDs numéricos. Pusimos un límite de 256 tokens (el 94.6% de las quejas entra en esa longitud, el resto se trunca). Las categorías de texto las pasamos a números (0 a 4): credit_card = 0, credit_reporting = 1, debt_collection = 2, mortgages_and_loans = 3, retail_banking = 4.

Para la división de datos usamos 80% para train, 10% para validación y 10% para test. Usamos muestreo estratificado, que básicamente significa que cada conjunto mantiene el mismo porcentaje de cada categoría que el dataset original. Así nos aseguramos de que la evaluación sea representativa.

4. Métricas de evaluación

Accuracy es el porcentaje de predicciones correctas sobre el total. Es fácil de entender pero con clases desbalanceadas puede dar una imagen engañosa (un modelo que diga siempre credit_reporting ya tendría 56% de accuracy sin haber aprendido nada).

Precision mide, de todas las quejas que el modelo puso en una categoría, cuántas realmente pertenecen a esa categoría. Es decir, qué tan confiable es cuando predice algo.

Recall mide, de todas las quejas que realmente son de una categoría, cuántas el modelo logró encontrar. Es decir, cuántas se le escaparon.

F1-score combina precision y recall en un solo número. La ventaja es que si uno de los dos es bajo, el F1 baja bastante, así que necesitamos que los dos estén bien para tener un buen F1. Por eso es la métrica que más miramos. Reportamos F1 macro (promedio simple entre categorías, las trata a todas igual) y F1 weighted (ponderado por cantidad de datos de cada categoría).

También usamos la **matriz de confusión** para ver en detalle dónde se equivoca: la diagonal muestra los aciertos y lo que queda afuera son los errores.

5. Resultados y discusión

5.1 Rendimiento general

En el conjunto de test obtuvimos **89.89% de accuracy** y **F1-score macro de 0.87**. Esto quiere decir que el modelo acierta casi 9 de cada 10 quejas. El F1 macro de 0.87 es importante porque al promediar todas las categorías por igual, nos confirma que el modelo no se limitó a andar bien solo en credit_reporting (que tiene muchos datos) sino que también funciona en las categorías chicas. Teniendo en cuenta que el dataset tiene un desbalance fuerte y que usamos un modelo relativamente chico (DistilBERT, 66 millones de parámetros) corriendo en Google Colab, que tiene restricciones de memoria y tiempo de uso, son números bastante buenos. De hecho, elegimos DistilBERT en vez de BERT completo justamente por eso: es más liviano y entra en esa GPU sin problemas de memoria.

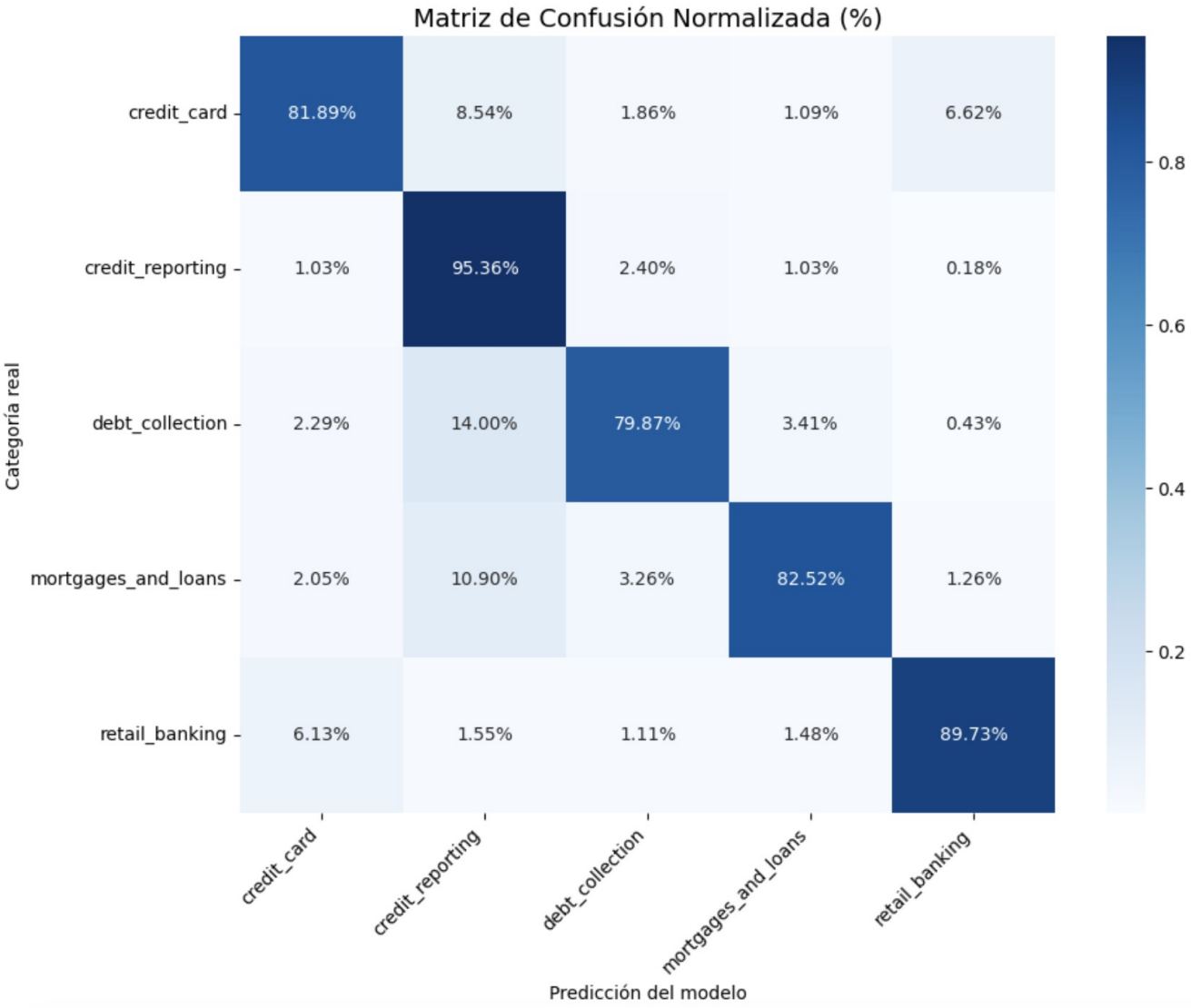
5.2 Resultados por categoría

Categoría	Precision	Recall	F1-score	Support
credit_reporting	0.93	0.95	0.94	9.118
debt_collection	0.85	0.80	0.82	2.315
mortgages_and_loans	0.88	0.83	0.85	1.899
credit_card	0.83	0.82	0.82	1.557
retail_banking	0.89	0.90	0.89	1.353
Macro avg	**0.87**	**0.86**	**0.87**	**16.242**
Weighted avg	**0.90**	**0.90**	**0.90**	**16.242**

Credit_reporting fue la que mejor anduvo (F1: 0.94), cosa esperable porque tiene muchos más datos que las demás. Lo que sí nos llamó la atención es retail_banking: con apenas el 8% de los datos llegó a 0.89 de F1. Eso nos dice que los pesos de clase hicieron su trabajo. Las categorías donde más le cuesta al modelo son credit_card y debt_collection (ambas con F1 de 0.82), probablemente porque usan un vocabulario bastante parecido entre sí.

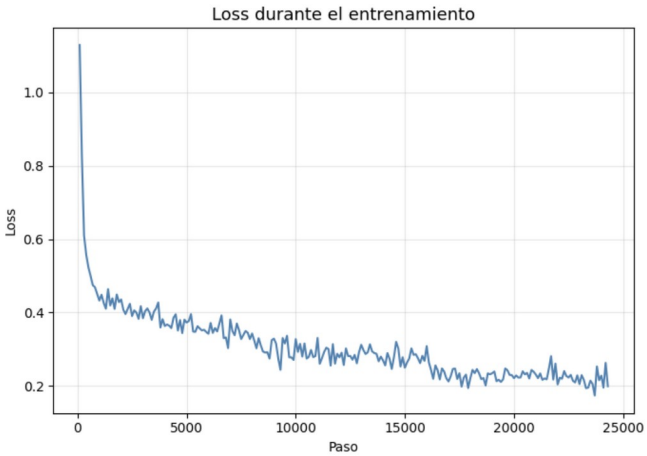
La diferencia entre macro avg (0.87) y weighted avg (0.90) es chica, lo que nos indica que el modelo no se limitó a andar bien solo en las categorías grandes. Si los pesos de clase no hubieran funcionado, esa diferencia sería mucho mayor.

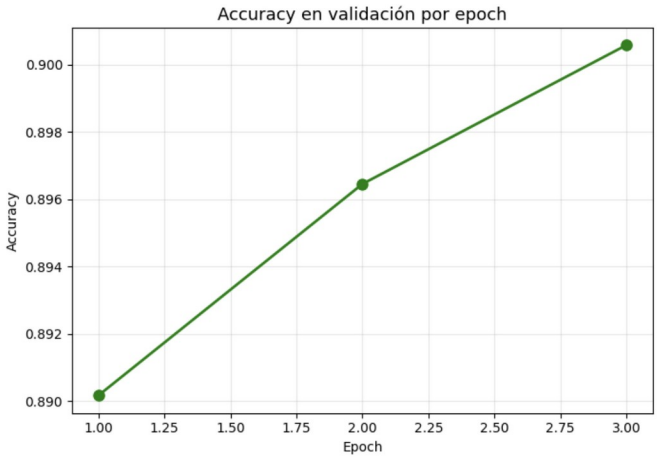
5.3 Análisis de confusiones



Mirando la matriz de confusión normalizada encontramos que los errores tienen bastante sentido. La confusión más grande es debt_collection clasificada como credit_reporting (14%). Esto pasa porque muchas quejas de cobro de deuda mencionan reportes de crédito, ya que están relacionados. Lo mismo con mortgages_and_loans que se confunde con credit_reporting (10.9%): las hipotecas afectan el historial crediticio y el vocabulario se mezcla. Credit_card se confunde con credit_reporting (8.5%) y con retail_banking (6.6%) porque al final son todos servicios bancarios y comparten terminología.

5.4 Overfitting





El loss de entrenamiento fue bajando de ~1.15 a ~0.20 durante las 3 epochs, con la caída más fuerte al principio. El accuracy en validación subió de forma constante: 89.0% en la epoch 1, 89.6% en la 2 y 90.0% en la 3. No vimos señales de overfitting en esta corrida, el accuracy nunca bajó de una epoch a la siguiente. De todas formas, dejamos activado load_best_model_at_end=True como precaución, para que si en alguna epoch el rendimiento cae, se quede con el mejor modelo automáticamente.

5.5 Dificultades y limitaciones

El desbalance de clases fue el primer problema que nos encontramos, aunque lo pudimos resolver bastante bien con los pesos de clase. La similitud de vocabulario entre categorías es una limitación más difícil de atacar, porque es inherente al problema: en la vida real las quejas de deuda y de reportes crediticios se parecen. El truncamiento a 256 tokens deja afuera parte de la información del 5.4% de quejas más largas, pero subir a 512 hubiera complicado el uso de memoria en Colab. También hay que tener en cuenta que DistilBERT trabaja solo con texto en inglés y en minúsculas, así que no sirve para quejas en otros idiomas.

6. Conclusiones

Con DistilBERT y fine-tuning logramos un clasificador que acierta casi 9 de cada 10 quejas, usando un modelo liviano que corre en Colab. Los pesos de clase fueron clave para que las categorías chicas no queden relegadas: retail_banking con solo 8% de los datos tuvo un F1 de 0.89. Los errores que comete el modelo se concentran en categorías que comparten vocabulario, lo cual es más una limitación del problema que del modelo.

Probamos early stopping con patience de 2 y un máximo de 10 epochs, pero el modelo siguió mejorando sin activar el freno. Como trabajo a futuro nos gustaría probar con bert-base-uncased para ver si un modelo más grande mejora los resultados, y subir el max_length a 512 tokens para capturar la información completa del 5.4% de quejas que se cortan. También sería interesante probar técnicas de data augmentation para las clases minoritarias.